COMPLETE SOURCE CODE ARCHITECTURE GUIDE

MANEKIN

FASHION & AI VIRTUAL TRY-ON PLATFORM

MANEKIN FASHION
全体コード完全解説ガイド

本ガイドは、オンラインファッションストア「MANEKIN FASHION」およびブラウザ完結型バーチャル試着室「MANEKIN AI STUDIO」を構成する **全15ソースファイル・全29ルート（API＋配信）・MySQL全10テーブル** の仕組みを、初心者にも全体像と処理の流れが立体的に理解できるよう体系化した技術解説書です。 全ファイルには1行ごとの日本語解説コメントも追記されており、本書とソースコードを並べて読むことで実装の細部まで追うことができます。

15	22,922	8,794	10 / 29
構成ファイル総数	総コード行数（実測）	初心者解説コメント行	MySQL表 / Flaskルート

本システムの6大コア機能と設計ポイント

1. シングルページ EC ストア：全24商品・MEN/WOMEN切替・カラー別画像・カートと注文完了まで1ページ遷移なしで動作

2. ポイント経済＆購入時クローゼット連動：初回1,000pt付与・1pt=1円割引・1%還元・注文確定と同時に対象者のクローゼットへ自動保存

3. アカウント完全二層分離：LocalStorageのユーザー固有キー（`_u_ID`）とMySQLの`user_id`フィルタ＆世代ガードで混在ゼロ

4. AI背景透過＆5カテゴリ形状ゲート：`@imgly/background-removal`と180×180 Canvas 10スライス解析＋MobileNet v2で非衣類を遮断

5. 5層ゴーストマネキン試着スタジオ：靴→ボトムス→トップス→アウター→帽子の重なり順とXS〜XL / 70〜140% 微調整・MySQL自動同期

6. デュアル動作（オンライン＋オフライン）：Python＋MySQL接続時はDB永続化、未接続時は`offlinePending`フラグ付きでローカル保持

対象ディレクトリ: MANEKIN2026-2 / データベース: manekin_db (MySQL)

作成: MANEKIN CODE GUIDE (実コード・実DB検査済み)

目次 & 本書の読み方

第1章 **システム全体構成・技術スタック・全15ファイル一覧**

ブラウザ（HTML/CSS/JS）× Python Flask（ポート8080）× XAMPP MySQL（ポート3306）の3層構造と、全15ファイルの役割・行数・コメント数を概観します。

第2章 **フロントエンド SPA 構造・CSS 設計・商品マスター**

10個のページビューをハッシュ（`#home` 〜 `#manekin-studio`）で切り替える `js/app.js`、5分割CSSの読み順、24商品の `AppState` 設計を解説します。

第3章 **EC ショッピング・ポイント精算・購入時自動クローゼット保存**

初期1,000pt・5,000円送料無料・利用ポイント上限・1%還元計算と、サーバー側 `POST /api/orders` トランザクションの4段階処理をシーケンス図で追います。

第4章 **アカウント完全分離設計（MySQL × LocalStorage 二層隔離）**

アカウント切替時に前ユーザーの履歴が残らない仕組み：`getScopedKey`、旧共有キー削除、非同期レース条件を防ぐ `_MutationSeq` 世代管理、400ms デバウンス同期を詳解します。

第5章 **MANEKIN AI STUDIO（5層試着・AI背景透過・5カテゴリ形状判定）**

z-index 5層マネキン、ブランド規格サイズ（XS〜XL）と手持ち服（70〜140%）、`@imgly` 透過、Canvas 10スライス二股検出＋MobileNet v2 判定ゲートを解説します。

第6章 **MySQL データベース（manekin_db）全10テーブル完全リファレンス**

実稼働データベースから取得した全10テーブルの ER 図、主キー・外部キー（CASCADE）・UNIQUE 制約、各カラムの型と意味を一覧化します。

第7章 **Python バックエンド（app.py）全29ルート完全リファレンス**

商品・注文・認証・お気に入り・コーデ・3Dマネキン・クローゼット・AI提案・カート・スタジオ状態の全 API エンドポイントを関数名・行番号付きで網羅します。

第8章 **運用スクリプト解説 & 初心者向け Web / DB / AI 用語集**

`start_server.bat`・`setup_db.py`・`clear_orders.py`・`patch_appstate.py` の役割と、コード頻出の基本用語（SPA・API・CORS・UNIQUE・デバウンス等）を整理します。

ソースコードとの対応について

各ファイルには初心者向けに `// ↓ ...`（JS）、`/* ↓ ... */`（CSS）、`<!-- ... -->`（HTML）、`# ↓ ...`（Python）、`REM ...`（バッチ）のコメントを1行レベルで挿入しています。 動作ロジック・DOM構造・数値定数は元の実装から1文字も変更していないため、本書に記載した関数名や行番号を手掛かりにそのままコードを読み進めることができます。

第1章 システム全体構成・技術スタック・全15ファイル一覧

MANEKIN FASHION は、ユーザーが目にする**フロントエンド（HTML / CSS / JavaScript）**と、データ保存や安全検証を担う**バックエンド（Python Flask）**、そしてデータを永続保管する**データベース（MySQL）**の3つの層が協調して動く Web アプリケーションです。

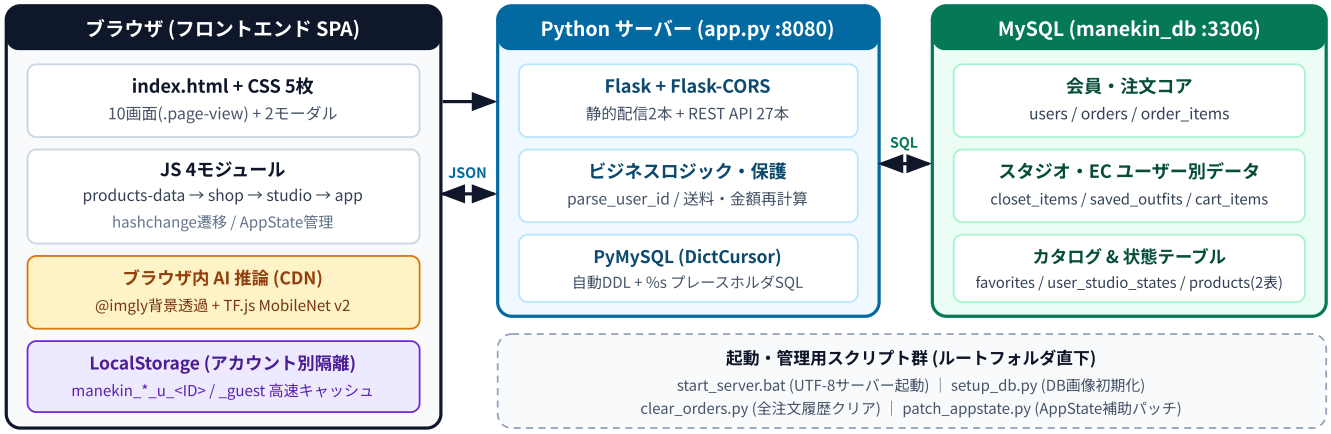


図1-1：MANEKIN FASHION 全体3層アーキテクチャとモジュール構成

全15ソースファイルの実測構成一覧

機能肥大化を防ぐため、CSS は用途別に5ファイル、JavaScript は責務別に4ファイルへ分割されています（以前の二重定義は解消済み）。

ファイルパス	分類	行数	コメント	Bytes	主な役割・含まれる中核機能
`index.html`	HTML(SPA)	2,531	897	189,752	全10画面の `.page-view` ・2モーダル・CDN AI読込・JS 4本読込
`css/style.css`	CSS入口	23	14	1,983	`@import` で下記4つの分割CSSを順序通りに一括読込
`css/base.css`	CSS基盤	839	368	38,845	`:root` デザイン変数、リセット、固定ヘッダー、ボタン、SPA表示切替
`css/shop.css`	CSS(EC)	3,111	1,214	111,278	HERO、商品カード、詳細、カート、ポイントカード、ログイン、マイページ
`css/studio.css`	CSS(試着)	4,242	1,837	128,192	3カラム試着室、5層 `.layer-slot` 、微調整パネル、AI診断カード
`css/responsive.css`	CSS(端末)	2,042	838	64,839	1100px / 992px / 768px / 480px ブレークポイント・スマホ最適化
`js/products-data.js`	JS(データ)	836	446	36,423	公式カタログ全24商品（MEN 12点 / WOMEN 12点） `SAMPLE_PRODUCTS`
`js/app.js`	JS(中核)	737	334	41,614	`AppState` 状態管理、 `handleRouting()` SPAルーター、起動初期化
`js/shop.js`	JS(EC機能)	2,816	935	154,166	アカウント別ストレージ管理、 `applyAccountSession` 、決済、ポイント、認証
`js/studio.js`	JS(試着室)	3,610	1,174	196,438	5層マネキン描画、サイズ調整、AI背景透過＆5カテゴリ形状判定、同期
`app.py`	Python(API)	1,871	629	105,939	Flask サーバー本体。33関数・29ルート、注文トランザクション、DB操作
`setup_db.py`	Python(保守)	141	52	11,247	初期7商品分の画像パスとカラーバリエーションをMySQLへ投入
`clear_orders.py`	Python(保守)	59	25	4,139	`order_items` → `orders` の順に全注文履歴を削除して初期化
`patch_appstate.py`	Python(保守)	30	14	2,058	`js/app.js` の `AppState` へ `selectedGender` を追加した補助パッチ
`start_server.bat`	Windows起動	34	17	2,561	UTF-8 (`chcp 65001`) 設定して `python app.py` を起動
合計（15ファイル）	**Full-Stack**	**22,922**	**8,794**	**1,089,474**	**合計約 1.09 MB（コメント率 38.4%、既存ロジック変更ゼロ）**

第2章 フロントエンド SPA 構造・CSS 設計・商品マスター

1. SPA（シングルページアプリケーション）の画面切り替え仕組み

一般的な複数HTMLサイトとは異なり、MANEKIN FASHION は `index.html` 1枚の中に全10画面（`<section class="page-view">`）を最初から定義しています。普段は CSS の `.page-view { display: none; }` で隠れており、URL末尾の「`#`（ハッシュ）」が変わるたびに `js/app.js` 内の `handleRouting()` が目的のビューに `active` クラスを付けて画面を切り替えます（ファイルリロードが無いため高速で状態が途切れません）。



図2-1：js/app.js による SPA ハッシュルーティングと認証ガード（数値は index.html / app.js の開始行）

2. JavaScript の厳格な読み込み順序 (index.html L2493-2527)

1. `<script type="module">`：CDN から `@imgly/background-removal` を動的読込し `window.imglyRemoveBackground` へ公開
2. **TF.js + MobileNet v2.1.0**：画像認識用の軽量AIモデルを CDN 読込
3. `js/products-data.js`：最初に全24着の `SAMPLE_PRODUCTS` 配列を定義
4. `js/shop.js`：ストレージ分離・カート・決済・ポイント・認証・マイページ関数を定義し `window.*` へ公開
5. `js/studio.js`：マネキン試着室・AI形状判定・クローゼット・同期関数を定義
6. `js/app.js`：全関数が揃った最後に `AppState`・`handleRouting()`・`DOMContentLoaded` 初期化を起動

3. CSS モジュール分割とデザインシステム (css/style.css)

`index.html` は入口の `css/style.css` だけを1本読み込み、内部の `@import` が以下の4ファイルを上から順に適用します。

- `css/base.css`：CSS変数（`--color-black: #111111`、`--header-height: 72px`、`--container-max-width: 1200px`）、ボタン、固定ヘッダー
- `css/shop.css`：トップHERO（85vh）、4列商品グリッド、レジ画面、ログイン画面、マイページ
- `css/studio.css`：3カラムスタジオ（310px / 1fr / 340px）、5層マネキン座標、微調整、AI判定カード
- `css/responsive.css`：必ず末尾に読み込まれ、**1100px / 992px / 768px / 480px** の画面幅別に上書き最適化

4. アプリ全体のメモリ状態 `AppState` (js/app.js L24-145) & 商品マスター 24着

`AppState` は画面全体で共有される「今この瞬間」の作業メモリです。`currentView`（現在画面）、`selectedGender / selectedCategory / selectedSort`（絞り込み条件）、`currentUser`（ログイン会員）、`pendingAuthTarget`（ログイン後の自動復帰先: `'manekin'` または `'checkout'`）、`cart / favorites / orderHistory / myCloset / savedOutfits / currentOutfit / fittingOffsets / studioGender` を一元管理します。 商品カタログ `SAMPLE_PRODUCTS` には、`id: 1〜12`（MEN 12着）と `id: 13〜24`（WOMEN 12着）の計 **24着** が登録され、それぞれ `colors[]`・`colorImages{}`・`sizes: ["S","M","L","XL"]`・`isNew`・`isFeatured` を持ちます。

第3章 EC ショッピング・ポイント精算・購入時自動クローゼット保存

本システムのショッピング決済（`js/shop.js` の `updateCheckoutCalculations()` / `handlePlaceOrder()` ＋ `app.py` の `POST /api/orders`）は、「ポイント計算」「改ざん防止」「購入後の試着比較連動」を単一トランザクションで同時に完結させる設計です。

ポイント制度の基本ルール

- **入会特典**：新規登録・デモ会員ともに初期 **1,000 pt** 付与
- **利用レート**：**1 pt ＝ 1 円** として値引き
- **還元率**：実支払額 `netTotal` の **1%**（`Math.floor(netTotal * 0.01)`、1円以上で最低 `1 pt` 保証）

配送料と合計金額の計算式

- **商品小計** `subtotal`：`Σ(単価 × 数量)`
- **配送料** `shipping`：小計 **5,000円以上 無料** / 未満 **550円**
- **利用上限**：`min(保有pt, subtotal + shipping)`
- **お支払い総額**：`netTotal = max(0, grossTotal - usedPoints)`

購入時クローゼット自動登録

- 注文完了と同時に MySQL の `closet_items` へ `is_owned=1, is_purchased=1` で自動保存
- ID命名規則：`purchased-{product_id}-{order_no}-{idx}`
- 過去注文の手動取込 `importOrderToCloset(idx)` も同じ規則を使用

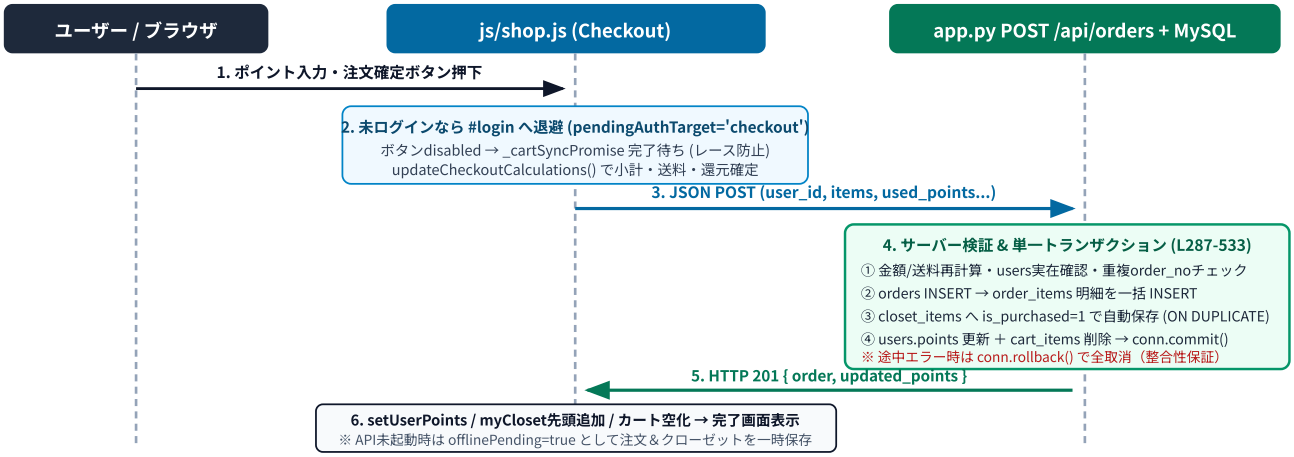


図3-1：購入確定ボタン押下から MySQL 確定・ポイント精算・自動クローゼット登録までの処理シーケンス

初心者メモ：なぜサーバー側（app.py）でも小計・送料を再計算しているの？

ブラウザ上の JavaScript は開発者ツール等で誰でも書き換えられるため、送信された合計額をそのまま信じるのは危険です。そのため `create_order()`（`app.py` L328-346）では商品リストから単価×数量と送料（5,000円条件）を再計算し、MySQL に正しい請求額が記録されるようにしています。また `sync_mode=True`（オフライン履歴の後追い同期）の場合は、二重にポイント増減やカート消去が起きないようステップ④をスキップします。

第4章 アカウント完全分離設計（MySQL × LocalStorage 二層隔離）

従来構成ではブラウザの `localStorage` に「`manekin_orders`」のような全ユーザー共通キーを使っていたため、別アカウントへログインし直しても前の人の購入履歴・カート・クローゼットが残ってしまう問題がありました。現在は**ブラウザ側・サーバー側の二層でアカウントごとの完全隔離**を行っています。



図4-1：アカウント間のデータ混在をゼロにするクライアント×サーバー二層分離とレース対策

なぜ「MutationSeq（世代カウンター）」と「400msデバウンス」が必要か？

- **非同期の追い越し上書き防止（世代カウンター）**：ログイン直後に裏で `syncOrdersFromBackend()` や `syncClosetFromBackend()` の GET 通信が走っている途中で、ユーザーが「即ログアウト」や「手持ち服追加」を行うと、古い GET 応答が後から返ってきて最新状態を巻き戻す恐れがあります。そこで開始時のユーザーIDと世代番号（`_ordersMutationSeq`、`_closetMutationSeq` 等）を照合し、1つでもズレていたら通信結果を破棄します。
- **スライダー連打の通信最適化（デバウンス & フラッシュ）**：試着室のスライダーを1px動かすたびに MySQL へ書き込むと負荷が大きいため、`syncStudioStateToBackend()` は **400ミリ秒** 待機してから最後の値だけを送信します。その代わりログアウトやユーザー切替が起きる直前には `flushPendingStudioStateSync()`（`js/studio.js` L3436）が待機タイマーをキャンセルし、必ず**旧ユーザーのID宛て**に即時確定させてからセッションを切り替えます。

第5章 MANEKIN AI STUDIO（5層試着・AI背景透過・5カテゴリ形状判定）

`js/studio.js`（3,610行）と`css/studio.css`（4,242行）が実現する試着スタジオは、「クロームスタンドSVG＋5層着せ替えステージ」「実サイズ連動マイクロ調整」「ブラウザ完結AI背景削除」「5カテゴリ限定AI形状ゲート」「リアルタイム色彩診断＆ルックブック」の5本柱で構成されています。

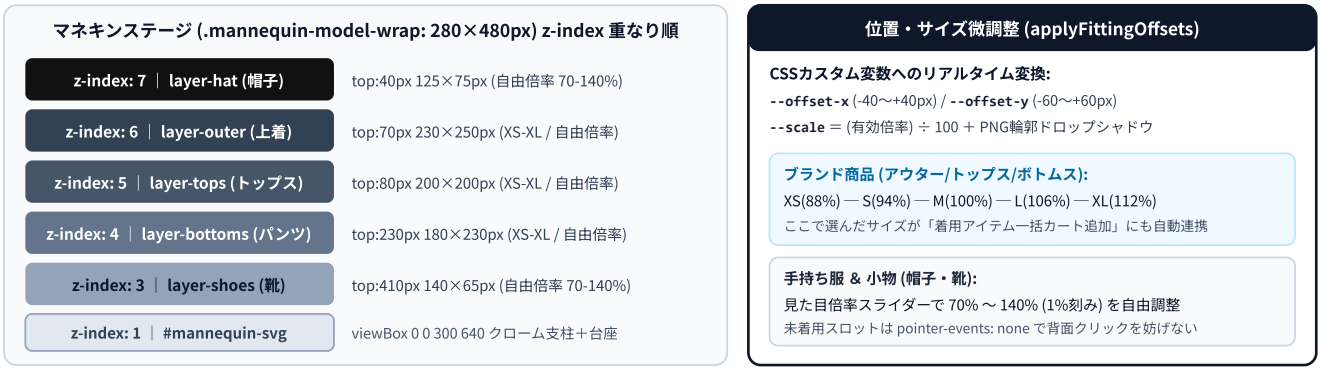


図5-1：ゴーストマネキン5層レイヤー重なり設計とサイズスケール連動機構

どんな写真でも登録できてしまう問題を解決：AI背景透過＆5カテゴリ限定形状ゲート

「アウター・ボトムス・トップス・靴・帽子」以外の写真（風景、犬・猫、食べ物、パソコン、書類スクショ等）がマイクローゼットへ登録されるのを防ぐため、モーダル側では**AI背景透過**に続いて**2段階の形状＆画像認識ゲート（`analyzeClothingItemShape` L2589-2956）**を実行し、`currentAiAnalysisResult.isValid !== false`でないと保存ボタンを押せない仕組みになっています。

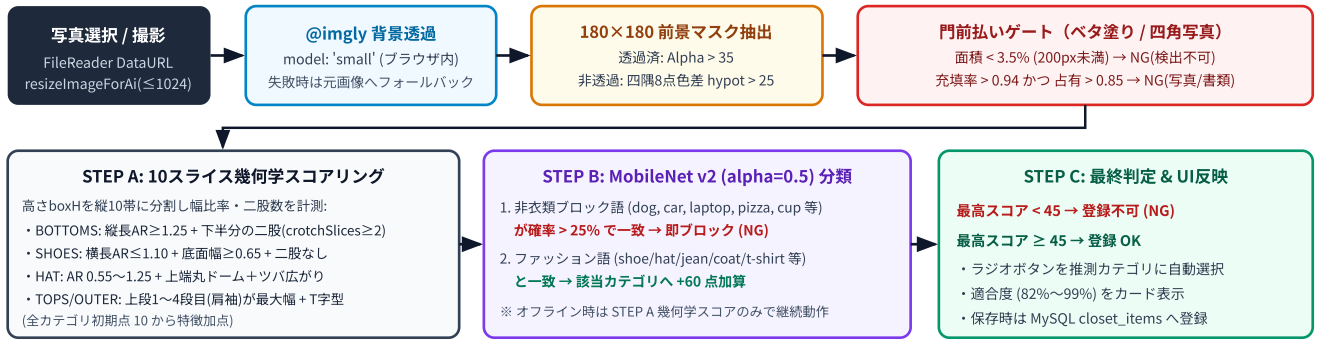


図5-2：ブラウザ内 AI 背景透過（@imgly）と 5カテゴリ限定・形状判定ゲートパイプライン

第6章 MySQL データベース（manekin_db）全10テーブル完全リファレンス

バックエンドは XAMPP 上の MySQL（`127.0.0.1:3306 / manekin_db`）へ接続します。以下のスキーマは稼働中の実 DB を直接 `DESCRIBE` / `SHOW CREATE TABLE` して検証した最新構成です（ユーザー要望により旧体型プロフィール列は `users` から削除済み）。`cart_items` と `user_studio_states` は `app.py` 起動時に `ensure_account_tables()` が自動保証します。

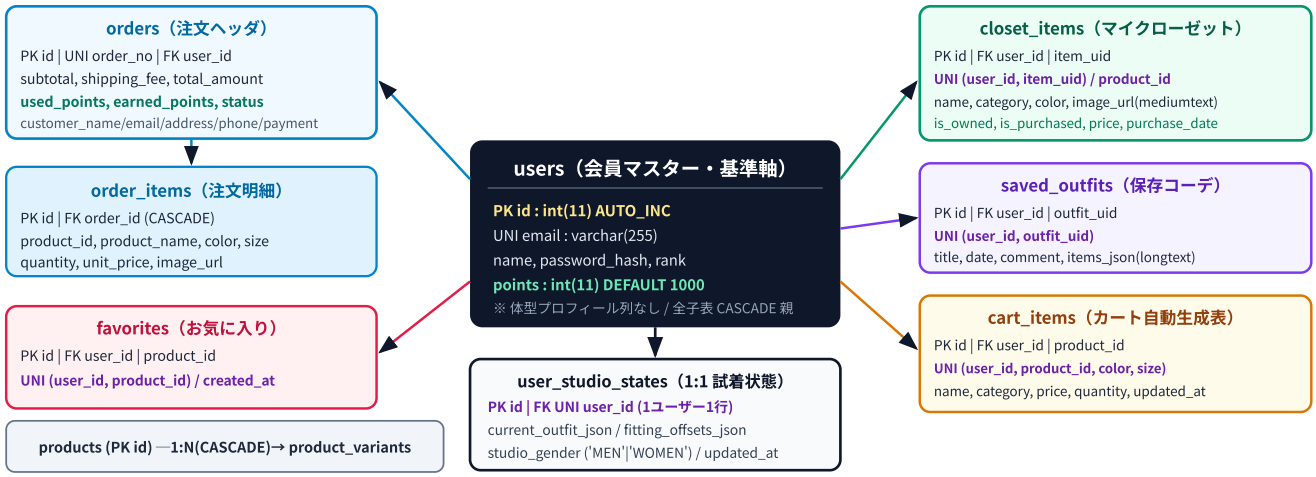


図6-1：MySQL (manekin_db) 全10テーブル実体関連図 (ER 図) — 外部キーはすべて ON DELETE CASCADE 保護

全10テーブル詳細定義一覧

テーブル名	キー / 制約（実DB定義）	主要カラム・型・役割
`users`	PK `id` UNI `email`	`id int(11)`, `name varchar(100)`, `email varchar(255)`, `password_hash varchar(255)`, `rank varchar(50) DEFAULT 'REGULAR MEMBER'`, `created_at datetime`, `points int(11) NOT NULL DEFAULT 1000`
`products`	PK `id`	`gender varchar(10)`, `name varchar(255)`, `name_en varchar(255)`, `category varchar(50)`, `price int(11) DEFAULT 0`, `description text`, `material varchar(255)`, `sku varchar(50)`, `is_new / is_featured tinyint(1)`, `image varchar(255)`
`product_variants`	PK `id` FK `product_id` →products	`product_id int(11)`, `color varchar(50)`, `color_code varchar(20)`, `image_url varchar(255)`（1商品のカラー展開ごとの画像URL・見本色を保持）
`orders`	PK `id` UNI `order_no` FK `user_id` →users	`order_no varchar(50)`, `user_id int(11)`, 配送先(`customer_name`, `customer_email`, `postal_code`, `address text`, `phone`), `payment_method`, `delivery_time`, `subtotal`, `shipping_fee`, `total_amount`, `status DEFAULT '注文受付済み'`, `used_points`, `earned_points`, `created_at`
`order_items`	PK `id` FK `order_id` →orders	`order_id int(11)`, `product_id int(11)`, `product_name varchar(255)`, `color varchar(50)`, `size varchar(10)`, `quantity int(11) DEFAULT 1`, `unit_price int(11) DEFAULT 0`, `image_url varchar(255)`
`favorites`	PK `id` FK `user_id` UNI `(user_id, product_id)`	`user_id int(11)`, `product_id int(11)`, `created_at datetime`（`INSERT IGNORE` やトグル追加/解除でアカウント別に管理）
`closet_items`	PK `id` FK `user_id` UNI `(user_id, item_uid)`	`item_uid varchar(100)`, `product_id int(11)`, `name varchar(255)`, `category`, `color DEFAULT 'BLACK'`, `season DEFAULT '通年'`, `image_url mediumtext`（透過Base64対応）, `is_owned`, `is_purchased`, `price`, `purchase_date`

テーブル名	キー / 制約（実DB定義）	主要カラム・型・役割
saved_outfits	PK id FK user_id UNI (user_id, outfit_uid)	outfit_uid varchar(100), title varchar(255), date varchar(50), comment text, items_json longtext（5部位の着用服JSONを丸ごと保存）, created_at datetime
cart_items	PK id FK user_id UNI uk_user_cart_item	product_id int(11), name, category DEFAULT 'TOPS', color DEFAULT 'STANDARD', size DEFAULT 'M', price, quantity DEFAULT 1, image_url text, updated_at（一意キー: user_id, product_id, color, size）
user_studio_states	PK id FK user_id UNI uk_studio_state_user	user_id int(11), current_outfit_json longtext, fitting_offsets_json longtext, studio_gender varchar(16) DEFAULT 'MEN', updated_at datetime ON UPDATE current_timestamp()

MySQL スキーマ設計のポイント（なぜこの型・制約を選んでいるのか）

1. 型サイズの適切な使い分け

メールや商品名は varchar(255) で検索・索引を高速化し、AI背景透過後の手持ち服画像（Base64 PNG で数十〜数百KBになる）には最大 16MB の mediumtext（closet_items.image_url）を割り当て、画像データ欠損を防いでいます。また 5部位JSON には longtext を採用しています。

2. 複合 UNIQUE による冪等更新

favorites(user_id, product_id)・closet_items(user_id, item_uid)・saved_outfits(user_id, outfit_uid) に複合一意制約を設けることで、通信リトライ時も同じ人の同じアイテムが二重登録されず、ON DUPLICATE KEY UPDATE で安全に上書きできます。

3. CASCADE と自動マイグレーション

全ユーザー従属テーブル（8表）に ON DELETE CASCADE を設定し孤立データを防止。また旧 users から不要な体型列を排除し、新設した cart_items / user_studio_states は app.py の ensure_account_tables() が起動時に自動保証します。

初心者メモ：orders と order_items を2つのテーブルに分ける理由（第1正規化）

1回の注文で3着の服を買う場合、orders 1行に無理やり「商品1, 商品2, 商品3...」と列を増やすと4着以上で破綻してしまいます。そこで「注文全体（誰が・いつ・何ポイント使って・合計いくらか）」を親の orders（1行）に持ち、「買った服ごとの明細（商品名・色・サイズ・単価・数量）」を子の order_items（N行）へ切り出すことで、何点同時購入しても美しいリレーションで集計できるようになります。

第7章 Python バックエンド（app.py）全29ルート完全リファレンス

`app.py`（1,871行）には全33個の Python 関数と **29個の Flask ルーティング（API 27本 ＋ 静的配信 2本）** が定義されています。ほぼすべてのデータ操作 API で `parse_user_id()` と SQL の `WHERE user_id = %s` によるアカウント境界チェックが行われます。

Method	エンドポイントパス	Python関数 / 行	処理内容・アカウント分離/入力ポイント
GET	`/api/products`	`get_all_products` (L168)	MySQLから全商品と `product_variants` 色別画像を結合して返却
GET	`/api/products/<int:product_id>`	`get_product_detail` (L219)	指定商品IDの詳細・カラー別画像・サイズ一覧を返却（無ければ404）
GET	`/api/health`	`health_check` (L270)	Flask稼働＆MySQL接続診断（`mysql: "connected"|"disconnected"`）
POST	`/api/orders`	`create_order` (L287)	注文検証・送料/総額再計算・`orders` / `order_items` / `closet_items` 登録・ポイント増減・カート空化
GET	`/api/orders`	`get_orders` (L538)	`?user_id=` の本人注文＋明細のみを最新順で取得（無効時は空配列）
DEL	`/api/orders`	`clear_orders` (L615)	`?user_id=` またはJSONの本人分 `order_items` → `orders` のみを削除
POST	`/api/auth/register`	`register_user` (L669)	メール重複(409)確認後、初期 **1,000pt** 付きで `users` 作成(201)
POST	`/api/auth/login`	`login_user` (L738)	`email` ＋ `password_hash` 照合し会員情報返却（失敗時は401）
POST	`/api/auth/demo`	`demo_login_user` (L788)	`demo@manekin.com`（VIP/1000pt）を検索し未登録なら自動作成して認証
POST	`/api/user/sync`	`sync_user_data` (L846)	保有ポイント値を指定 `user_id` の `users.points` へ同期更新
GET	`/api/favorites`	`get_favorites` (L893)	`?user_id=` のお気に入り `product_id` 重複なし配列を取得
POST	`/api/favorites/toggle`	`toggle_favorite` (L930)	`(user_id, product_id)` が有れば DELETE(removed)、無ければ INSERT(added)
POST	`/api/favorites/sync`	`sync_favorites` (L980)	`INSERT IGNORE` でローカルのお気に入りID配列を一括同期
GET	`/api/outfits/<int:user_id>`	`get_user_outfits` (L1034)	`saved_outfits` から本人保存コーデを取得し `items_json` を復元
POST	`/api/outfits/save`	`save_user_outfit` (L1082)	コーデとAIコメントを `ON DUPLICATE KEY UPDATE` で保存/更新(201)
DEL	`/api/outfits/delete/<outfit_uid>`	`delete_user_outfit` (L1145)	`outfit_uid = %s AND user_id = %s` の条件で本人所有分のみ削除
GET	`/api/manekin/avatar/<user_id>`	`get_manekin_avatar` (L1204)	3Dマネキン拡張用 `DEFAULT_AVATAR_PROFILE`（身長172cm等）返却
POST	`/api/manekin/try-on`	`manekin_virtual_try_on` (L1216)	仮想試着プレビュー＆フィット診断（`OPTIMAL_FIT`）のAPIスタブ返却
GET	`/api/closet/<user_id>`	`get_user_closet` (L1246)	`closet_items` から本人所有アイテム一覧を新しい順(id DESC)で取得

Method	エンドポイントパス	Python関数 / 行	処理内容・アカウント分離/入力ポイント
POST	`/api/closet/add`	`add_to_closet` (L1287)	アップロード服や購入商品を `closet_items` へ保存/重複時更新(201)
DEL	`/api/closet/delete/<item_uid>`	`delete_closet_item` (L1378)	`item_uid = %s AND user_id = %s` で本人手持ち服のみ安全に削除
POST	`/api/ai-coordinate/recommend`	`recommend_coordinates` (L1422)	天気・気温・シーンを受け取り推奨コーデ＆信頼度0.94を提案
GET	`/api/cart/<int:user_id>`	`get_user_cart` (L1462)	`cart_items` テーブルから指定アカウントのカート商品を登録順に取得
POST	`/api/cart/sync`	`sync_user_cart` (L1518)	同一(商品/色/サイズ)の数量を合算正規化し「全削除→全挿入」で厳密置換
DEL	`/api/cart/<int:user_id>`	`clear_user_cart` (L1629)	指定アカウントの `cart_items` 行のみを全クリア
GET	`/api/studio-state/<int:user_id>`	`get_user_studio_state` (L1677)	`user_studio_states` から着用5部位・微調整値・性別を取得（未保存時は初期値）
POST	`/api/studio-state/save`	`save_user_studio_state` (L1758)	現在の試着状態・微調整オフセット・性別を `ON DUPLICATE KEY UPDATE` で保存
GET	`/`	`serve_index` (L1825)	トップアクセス時に `index.html` をブラウザへ返却
GET	`/<path:path>`	`serve_static` (L1832)	`css/*`・`js/*`・`assets/*` などの静的ファイル一式を配信

app.py バックエンド実装における安全性・運用設計

SQLインジェクション対策 & フェイルクローズ

- **パラメータ化クエリ**：全 SQL 文で文字列結合を禁止し、`cursor.execute("... WHERE user_id = %s", (uid,))` 形式で値を安全にエスケープします。
- **不正IDのフェイルクローズ**：`parse_user_id()`（`app.py` L81）が文字列・負数・`0`・`None` を一律 `None` として弾き、未認証リクエストへ他人のデータが絶対に出ない構造です。

HTTPステータス設計 & サーバー起動仕様

- **意味のあるステータス返却**：作成成功 `201`、取得/更新 `200`、入力不備 `400`、認証失敗 `401`、未存在 `404`、メール重複 `409`、例外ロールバック `500` を厳密に返します。
- **ポートとCORS設定**：`PORT = int(os.environ.get("PORT", 8080))` で自動待受し、開発中のブラウザ直接アクセスに備えて `CORS(app)` を有効化しています。

第8章 運用補助スクリプト & 初心者向け Web / DB / AI 用語集

1. サーバー起動・保守・補助スクリプトの使い方

ファイル名	実行コマンド	動作仕様と使うタイミング
`start_server.bat`	ダブルクリック / 実行	コマンドプロンプトを UTF-8 (`chcp 65001`) に切り替え、自身のディレクトリへ移動して `python app.py` を起動します。エラー時は必要パッケージ（ `pip install flask flask-cors pymysql` ）の案内を出して停止します。
`setup_db.py`	`python setup_db.py`	MySQL に `manekin_db` が無ければ作成し、初期7商品の代表画像パス更新と `product_variants` 19件の再投入を行います（旧テスト商品 ID:8 が残っていれば自動削除）。
`clear_orders.py`	`python clear_orders.py`	外部キー制約エラーを防ぐため、必ず子テーブル `order_items` を先に `DELETE` し、次に親テーブル `orders` を空にして注文履歴を0件へリセットします。
`patch_appstate.py`	開発時補助パッチ	`js/app.js` の `AppState` に `selectedGender: "ALL"` を1回だけ挿入するために作成した開発履歴スクリプトです（適用済み環境では `"Target not found"` を出力し安全に終了します）。

2. はじめてコードを読む人のための重要キーワード用語集

SPA (シングルページアプリケーション)
HTMLページを読み直し直さず、1つの `index.html` 内で JavaScript が表示領域（ `#home` 〜 `#manekin-studio` ）を書き換えて画面遷移に見せる仕組み。画面状態（試着中コーデ等）を保ったまま高速に移動できます。

REST API & JSON (`fetch` 通信)
ブラウザが裏側から Python サーバーのURL（ `/api/...` ）へリクエストを送り、文字データの共通フォーマット「JSON」でデータを受け取る仕組み。ページを再読み込みせずにDB保存できます。

LocalStorage スコープ隔離
ブラウザ内にデータを保存する仕組み。本アプリでは `_u_7` のようにユーザーIDを末尾に付けて保存箱を分け、ログイン切替時のデータ混線を防いでいます。

トランザクション (`commit` / `rollback`)
注文処理のように「注文挿入・明細挿入・ポイント引き落とし・カート空化」の複数を「全部成功」か「1つ失敗したら全部やり直し」にする DB の安全機能です。

外部キー & `ON DELETE CASCADE`
`orders.user_id` のように別テーブルの親IDと紐づく列。 `CASCADE` が付いているため、親会員を削除すると関連するカートや注文明細も漏れなく連動削除されます。

Upsert (`ON DUPLICATE KEY UPDATE`)
同じ `UNIQUE` キーのデータが無ければ新規登録（INSERT）、すでに存在すればエラーにせず最新値で上書き更新（UPDATE）する便利な MySQL 構文です。

デバウンス (Debounce) & 世代ガード
スライダー連続操作時に「最後の変更から400ms待つ」ことで通信回数を減らす工夫がデバウンス。通信の追い越し上書きを数字の連番で防ぐ仕組みが世代ガード（ `_MutationSeq` ）です。

WASM / ブラウザ完結型 AI 推論
`@imgly/background-removal` （背景切り抜き）や TensorFlow.js MobileNet v2（物体認識）は、外部サーバーへ写真を送らず自分のブラウザ内だけで画像解析を実行します。

コード検証・動作確認ステータス（実環境テスト済み）

本書の解説対象である全15ファイルへのコメント追記後、全ファイルの構造・トークン一致非破壊テスト（15 / 15 PASS）および Python＋MySQL 実接続でのヘッドレス Chrome E2E 動作検証（全ハッシュルート巡回・デモログイン・5層マネキン・タブ切替・微調整パネル・手持ち服登録モーダルAI診断・性別切替・ログアウト遷移／JS例外・HTTPエラー・コメント文字表示漏れ すべて 0件）を完了しています。